

研究进展简报

更新世界状态不等于 重新绑定操作目标

工具型智能体中的时序指称完整性 (Temporal Referent Integrity, TRI)

一句话概括 Agent 刷新外部状态时，世界事实可以更新，但先前已经由用户语义确定的操作对象不能在未经授权的情况下被悄悄换成新的选择器赢家。

拟投稿	AAAI 2027 Main Technical Track
当前主稿	8 页：7 页正文 + 1 页参考文献
核心方法	刷新前目标承诺编译 (Compile-then-act, CTA)
证据范围	受控诊断、多模型复现、SQLite 写入、人类验证、外部审计
更新时间	2026 年 7 月 21 日

供内部讨论使用 | 重点：研究思路、实验闭环、已有结果、投稿风险

执行摘要

当前判断 论文的核心闭环已经完成，现阶段最重要的不是继续堆同质实验，而是把问题新颖性、与 Binding Drift 的区别、外部零结果以及方法适用边界讲清楚。

这篇论文关注工具型 Agent 在状态更新之后出现的一类目标身份错误。Agent 先根据用户描述正确选中对象 A，刷新邮箱、数据库或任务列表后，对象 B 成为同一选择条件下的新赢家；模型虽然仍能看到 A 的 ID 和完整历史，却可能重新执行旧选择条件，把后续操作转移到 B。论文将这种“正确绑定之后、未经授权的目标替换”定义为时序指称完整性（TRI）问题。

研究设计的关键是构造语义上对称的最小对：同样的刷新和状态变化，在“现在选中，刷新后操作它”中必须保持 A；在“先刷新，再选择并操作”中必须改选 B。因此，永远锁定旧目标与永远刷新后重选都不成立，控制器必须显式判断指称是否已经成为身份承诺。

最重要的五个结论

- 现象可复现：在 160 条冻结主实验中，普通结构化状态只有 64.4%/71.9%，而 CTA 达到 95.0%/96.2%。
- 机制可归因：只增加 mode 字段、普通提前规划或有效性 gate 均不足；主要收益来自刷新前把“保持还是重选”编译成明确承诺。
- 结果可复制：在 240 条新状态、新 schema 上，Qwen、GLM、DeepSeek 的普通控制器分别出现 43/72、38/80、59/79 次条件漂移，CTA 均为 0。
- 错误有真实后果：上述核心漂移可重放为 SQLite 中的错误实体写入；六组完整历史基线的 1,440 次重放中共出现 414 次 wrong writes。
- 主张必须克制：公开 ToolSandbox、AppWorld、tau3 几乎没有原生严格 TRI 机会，低干预外部工具闭环为零结果；论文是机制诊断和评测盲区研究，不是现实发生率或通用安全保证。

关键数字速览

证据模块	规模	最关键结果
v3 主实验	160 条, 2 模型	Generic 64.4/71.9; CTA 95.0/96.2

证据模块	规模	最关键结果
v7 独立复制	240 条, 3 模型	条件漂移 43/72、38/80、59/79; CTA 为 0
强完整历史基线	1,440 条重放	普通/强提醒均仍有大量替换; 共 414 次 wrong writes
人工语义验证	3 人盲标 100 条	Fleiss kappa=.708; 多数票-gold 86%
独立人工改写	50 条, 2 模型	Generic 60/74; CTA 90/98
公开基准审计	129/244/2,449 家族或任务	严格原生机会几乎为 0; 支持 coverage blind spot

1. 研究问题：状态更新为何不等于目标重选

1.1 一个最小例子

考虑两条只在语序上不同的请求。刷新前邮件 A 是优先级最高的未读邮件，刷新后邮件 B 变成最高。

1. “现在找出优先级最高的未读邮件。刷新邮箱，然后回复它。”
2. “先刷新邮箱。然后找出优先级最高的未读邮件并回复。”

第一条请求在刷新前已经把“优先级最高的未读邮件”解析为 A，后面的“它”指向 A；刷新只更新邮箱事实，不授权改成 B。第二条请求则把选择动作有意推迟到刷新后，因此应该选择 B。两条任务经历完全相同的世界变化，却要求相反的目标。

核心矛盾 最新世界状态决定“现在谁满足选择条件”，但用户语义决定“后续动作是否仍指向先前已经解析的对象”。这两个问题不能由同一个“总是使用最新结果”的策略代替。

1.2 论文定义的错误

TRI 只把一类严格条件下的目标替换计为核心错误：

- 初始绑定正确：Agent 在刷新前已经找到了用户当时指向的对象；
- 旧对象刷新后仍存在，且对目标动作仍然有效；
- 刷新后同角色的另一个对象成为选择器的新赢家；
- 用户没有授权重选，但 Agent 最终把操作转移到新赢家。

这一定义主动排除了首次选错对象、对象被删除、动作前置条件失效、工具顺序错误、API 失败等其他问题。严格条件分母不是为了“挑结果”，而是为了回答错误是否真的发生在正确绑定之后。

1.3 与几个相邻问题的区别

相邻问题	它主要问什么	TRI 的不同点
初始实体绑定错误	第一次是否选对对象	先条件化到初始绑定正确，再研究刷新后是否有权换对象
记忆/上下文丢失	旧信息是否还在	Generic 已保留 ID、快照和完整指令，仍可能重新解释选择器
普通共指消解	文本中的代词指向谁	关注外部状态跨时间变化时，指称是否被授权转换
动作有效性	旧对象还能否执行动作	对象仍有效也可能被错误替换；有效性不等于身份授权
Binding Drift	实体是否在工作流中漂移、如何复核	加入 Preserve/Reevaluate 对称任务，核心是绑定后的转换授权

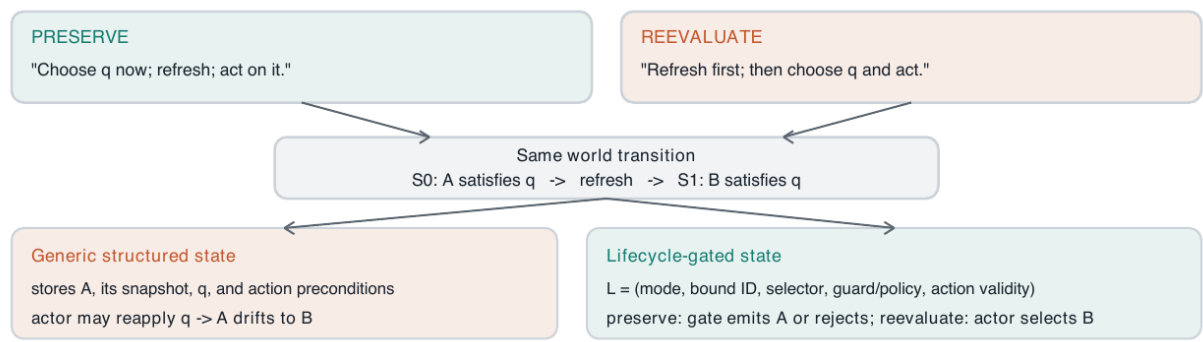


图 1 论文机制图：同一世界变化在 Preserve 与 Reevaluate 语义下要求不同目标；生命周期状态将这一差异变成可执行约束。

2. 方法思路：刷新前目标承诺编译

2.1 为什么普通结构化状态仍不够

普通 Structured Ledger 并不是刻意删除信息的弱基线。它保存原始指令、刷新前选择出的 ID、实体快照、选择条件、动作和动作前置条件，再让下游模型在刷新后自由决策。问题在于，这些字段只说明“过去发生过什么”，没有明确说明选择器现在是身份来源的历史记录，还是仍可执行的查询。模型可能看到所有证据，却仍把旧选择条件应用到新状态。

2.2 CTA 的处理流程

1. 刷新前理解时间语义：判断用户是在当前状态中已经选定对象，还是有意把选择留到刷新后。
2. 编译控制状态：Preserve 保存稳定实体 ID；Reevaluate 保存未解析的选择条件。
3. 刷新外部环境：更新事实，但不自动改写已经形成的身份承诺。
4. 执行动作：Preserve 使用原 ID；Reevaluate 在新状态中执行选择器；目标失效时按明确 fallback 拒绝或重选。

方法定位 CTA 的“编译”不是生成代码，而是把自然语言中的时间关系提前转换为一个明确的目标身份状态。它是简单的控制器机制，不是通用规划器。

2.3 类型化 Lifecycle 与确定性 gate

论文进一步实现了类型化生命周期记录，包括 reference mode、bound target ID、selector、action-relative validity 和 invalidity policy。确定性 gate 在 Preserve 分支检查已绑定对象是否仍满足动作前置条件：有效则发出原 ID，失效则按策略拒绝或进入授权的 fallback。实验显示 gate 只比自由 actor 多带来 1.2--1.9 个百分点，主要收益仍来自刷新前承诺编译。类型化状态的价值更多是可审计性、策略表达和执行边界，而不是主准确率来源。

2.4 理论直觉

对于状态完全相同的 Preserve/Reevaluate 最小对，如果一个刷新后策略只看到旧 ID、选择器、新状态和动作，却看不到原始语义或已编译的转换授权，那么两条任务对它的输入完全相同。确定性策

略只能输出同一个目标，因此至少会错一条。Always-Lock 和 Always-Reevaluate 正是这两个极端：一个保住保持型任务却破坏重选型任务，另一个相反。

3. 实验设计：如何把机制与其他错误分开

3.1 受控最小对与状态变化

每个领域定义实体表、自然语言选择器、排序或布尔条件、目标动作和动作前置条件。生成器在 S0 求得初始对象，施加受控状态变化，再在 S1 求得新赢家。配对任务共享领域、S0、S1、选择器、动作和 schema，只改变语序与指称方式。

变化类型	诊断作用
Flip	旧对象仍有效，但刷新后另一个对象成为选择器赢家；最直接测试未授权替换
Stable	赢家不变；检测控制器是否对任何刷新都过度反应
Remove	旧对象被删除；测试失效策略，不属于最纯粹的核心漂移
Invalidate	旧 ID 仍存在但动作前置条件失效；区分存在性和可执行性
Name collision	新对象获得相同显示名；测试是否依赖稳定 ID 而非表面名称

3.2 证据链与冻结纪律

实验层次	设计	要回答的问题
主发现	v3: 160 条，20 个语言模板簇，8 个领域， 2 模型	普通结构化状态是否会发生目标漂移；CTA 是否有效
独立复制	v7: 240 条，40 个新状态簇，10 个新 schema，3 模型	是否依赖原状态、原 schema、原模型
强基线	普通完整历史、刷新后一次性强语义提醒	是否只是 ledger 设计人为诱发；完整历史是否足够

实验层次	设计	要回答的问题
真实后果	模型面对 SQLite + 确定性写入重放	错误是否落实为错误实体修改
构念验证	3 人盲标 100 条 + 50 条独立英文改写	人类是否认同 Preserve/Reevaluate；是否依赖模板
外部边界	ToolSandbox、AppWorld、tau3 审计与自定义工具闭环	公开 benchmark 是否覆盖机会；受控结果能否自然外推
组合压力	多刷新、多角色、role indexing	单目标方法能否直接组合

主实验在模型调用前冻结数据、runner、hash、推理设置和不根据输出修改 prompt 的规则。主要置信区间以完整模板簇或状态簇为重采样单位，使用 10,000 次 cluster bootstrap；任务级 McNemar 只作为次要统计。API 失败、缺失任务、重试和解析错误单独审计，不解释为模型行为。

4. 主要实验结果

4.1 160 条主实验：普通记录不等于可执行承诺

控制器	Qwen3.5	GLM-5.1	解释
Generic free actor	64.4%	71.9%	信息完整，但目标身份约束隐式
Generic + mode	75.0%	75.0%	单独增加保持/重选标签，收益有限
Generic + validity gate	65.0%	73.1%	动作有效性不能阻止换到另一个有效对象
Untyped pre-refresh plan	81.2%	70.6%	提前规划本身不稳定
Exact CTA	95.0%	96.2%	刷新前编译目标承诺
Lifecycle free actor	96.9%	98.1%	类型化状态但无确定性 gate
Lifecycle + gate	98.1%	100.0%	gate 只额外增加 1.2--1.9 点

Generic 已保存关键事实，仍有约三成任务失败；CTA 将准确率提高到 95% 以上。Qwen 的生命周期方法把保持型任务从 33.8% 提升到 100%，同时重选型任务保持在约 95%--96%；GLM 在两类任务上都提高。这说明收益不是来自“永远保留旧 ID”。

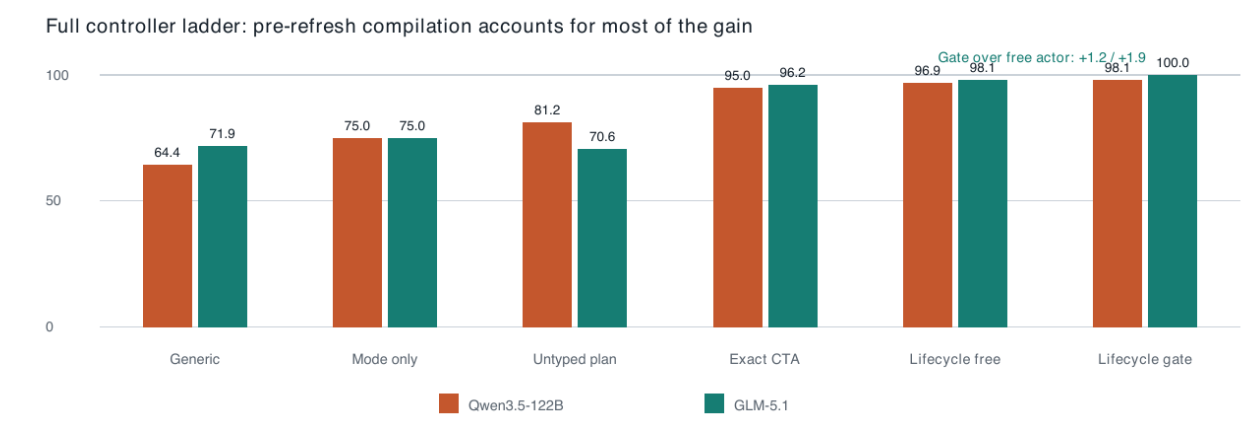


图 2 主实验控制器阶梯：Exact CTA 已解释大部分收益，生命周期 gate 的边际提升较小。

4.2 对称策略控制：两个极端都只有 60%

Always-Lock + validity 与 Always-Reevaluate 在 160 条任务上都得到 96/160（60%），但错误方向完全相反。Always-Lock 在保持型任务上为 80/80，在重选型上只有 16/80；Always-Reevaluate 则为 16/80 和 80/80。这个控制直接证明，论文要解决的不是锁定问题，而是语义条件下的选择性不变性。

4.3 240 条独立复制：三模型均观察到条件漂移

模型	Generic 准确率	CTA 准确率	Generic 条件漂移	CTA 条件漂移	CTA-GEN 差值
Qwen3.5	47.5%	70.8%	43/72	0	+23.3 点
GLM-5.1	70.0%	94.2%	38/80	0	+24.2 点
DeepSeek	73.8%	91.2%	59/79	0	+17.5 点

这里的分母随模型变化，是因为条件漂移只统计该模型已经正确完成初始绑定的机会。三模型的 Generic 合计出现 140 次核心漂移；CTA 在相应严格机会中均为 0。逐领域和逐模板家族剔除后，CTA 相对 Generic 的差值在三模型上始终为正，说明结果不是由某一个领域或模板主导。

4.4 强完整历史基线：历史保留和刷新后提醒仍不够

模型	普通完整历史	刷新后强语义提醒	CTA	CTA 相对强提醒
Qwen3.5	63.3%	69.6%	70.8%	+1.2, 区间跨 0
GLM-5.1	67.1%	80.8%	94.2%	+13.3
DeepSeek	68.8%	75.8%	91.2%	+15.4

刷新后的强提醒明确要求模型判断保持、重选或拒绝，确实比普通完整历史更好，但在每个模型的 80 条“保持型且刷新后赢家改变”任务中，仍分别把目标替换了 56、38、42 次。CTA 在 GLM 和 DeepSeek 上明显更强；在 Qwen 上总体准确率与强提醒持平，因此论文不能宣称 CTA 在所有模型上都显著优于最强后置提示。

统计口径提醒 完整历史基线没有独立、可审计的刷新前绑定事件，因此其目标替换只能称为无条件 substitution，不能直接计为 conditional TRI。

4.5 错误会变成真实数据库写入

模型	普通完整历史 wrong writes	强语义提醒 wrong writes	减少量
Qwen3.5	87/240	70/240	17
GLM-5.1	79/240	46/240	33
DeepSeek	75/240	57/240	18

模型	普通完整历史 wrong writes	强语义提醒 wrong writes	减少量
合计	241	173	68; 总计 414 次错误写入

六组输出共 1,440 个 episode，全部可重放，且没有最终 API 或解析失败。错误不是仅在文本中生成了错误 ID，而是修改了 SQLite 中不应被修改的记录。更严格的 v7 条件审计中，Qwen/GLM/DeepSeek Generic 的 43、38、59 次核心漂移也都成为 wrong-entity writes。需要同时说明：CTA 仍可能因 mode、初始 ID 或 selector grounding 错误产生非核心 wrong writes，0 条件漂移不等于 0 工具风险。

4.6 人类语义验证与独立改写

验证	规模	结果	解释
盲标一致性	3 人 × 100 条	Fleiss kappa=.708; alpha=.709	整体具有较高可操作一致性
多数票与 gold	100 条	86%; 94 条确定多数中为 91.5%	benchmark 语义大体符合人类判断
保持/重选核心	可执行保持 + 动态	保持可执行 86.7%; 动态 98%	论文核心语义最稳定
失效后 Reject	20 条	多数票-gold 55%; 一致 25%	fallback 是规范策略，不宜包装成人类共识
独立英文改写	50 条 × 2 模型	Generic 60/74; CTA 90/98	结果不只依赖作者模板

5. 外部验证与负面边界

5.1 公开 benchmark 的结论是“机会稀缺”，不是“发生率高”

对象	审计范围	严格原生 TRI 机会	应如何解释
ToolSandbox	129 个语义家族	0; 1 个 near-match	原生任务很少在正确绑定后插入可竞争的状态变化
AppWorld	732 任务 / 244 家族	0; 1 个更强 near-match	公开轨迹支持自然问题结构，但未观察到后绑定替换
tau3	2,449 任务 / 10,832 轨迹	0	released 轨迹再多也不能估计一个任务清单中没有的机制

现有任务通常缺少六个要素中的一个或多个：先前同角色绑定、独立状态变化、刷新后竞争赢家、稳定 ID、后续可执行写入、可评分的错误后果。因此外部审计最稳妥的贡献是指出 benchmark coverage blind spot，而不是声称现实中 TRI 普遍。

5.2 ToolSandbox-compatible 小规模正例

在冻结的 24 条自定义干预任务中，保留 ToolSandbox 原生 reminder 数据库和查询/修改工具，只加入 TRI 所需的任务语义与同步变化。严格 post-hoc 审计发现：GLM Generic 在 6 个机会中有 3 次未授权换目标，Stable 控制为 0/2；Qwen 的自由 Lifecycle actor 为 2/6，确定性 gate 重放后为 0/6。这个结果证明在真实工具接口中制造出严格机会时，现象可以发生，但任务仍是自定义的，且审计发生在输出之后，只覆盖 6 个状态簇。

正确表述 这是 benchmark-compatible intervention evidence，不是官方 ToolSandbox 排行榜结果，也不是确认性的外部发生率估计。

5.3 更低干预的外部闭环为零结果

在 96 条 ToolSandbox-style 扩展中，Qwen/GLM 完整历史与匹配 Generic 控制器在 70/73 和 64/87 个可判定机会里均为 0 条件 TRI；wrong writes 来自初始 selector 或工具顺序错误。两应用的低干预 AppWorld 闭环中，28 次正确及时绑定后也为 0 条件 TRI，两个真实 wrong writes 均发生在首次绑定之前。论文保留这些零结果，以限制“所有模型、所有工具环都会出现 TRI”的错误外推。

5.4 组合性与方法升级的负结果

两次刷新、多角色和多指称压力测试中，标量 Lifecycle 没有稳定胜过 Generic。角色索引将 Qwen 从 35/40 提升到 39/40，但 GLM 上与简单方法持平。Event Graph 和 Executable Selector 的 20 条 smoke 在两个模型上方向冲突，也没有达到预设门槛，因此未升级为主方法。这些结果把当前方法范围限制在控制器编排、单刷新、单操作目标的标量任务。

6. 论文最终可以主张什么

建议主张 在正确初始绑定之后，普通模型状态和完整历史仍可能把刷新后的选择器赢家替换为操作目标；刷新前编译目标承诺能够在受控标量任务中显著减少这一错误。该现象和收益依赖模型、控制器与机会结构。

6.1 证据支持的主张

- 世界事实刷新与操作目标重绑定是两个独立决策；同一状态变化下 Preserve 与 Reevaluate 可以要求相反目标。
- 结构化保存事实并不自动形成可执行的身份承诺；多个模型在正确绑定后仍会重新应用旧选择器。
- 刷新前承诺编译是主准确率增益的主要来源，类型化 Lifecycle 与 gate 主要增加可审计性和执行保护。
- 在严格条件机会中观察到的漂移会落实为错误实体写入，而不是只影响文本答案。
- 当前公开 Agent benchmark 对这种后绑定转换授权的覆盖非常有限。

6.2 不能主张的内容

- 不能说 TRI 在真实部署、自然用户请求或所有 Agent 中高频存在。
- 不能说所有 LLM 都会出现 TRI，或 CTA 在所有模型上都显著优于最强提示。
- 不能说 gate 提供通用写安全；错误 mode、错误初始 ID、selector grounding 与工具顺序仍会造成错误。
- 不能把 Always-Lock 直接包装成完整 Binding Drift Entity Lock 复现，也不能把强后置提示称为官方 self/cross reverify。
- 不能把低一致性的 Reject fallback 描述为人类共享语义；它更接近规范性 action policy。

7. 与 Binding Drift 的关系

Binding Drift 是最需要正面处理的邻近工作。两者都关注多步工具 workflows 中实体身份的稳定性，审稿人很可能质疑 TRI 是否只是 entity locking 的重新表述。论文当前的有效区分应放在二维问题地图上：第一个维度是初始绑定是否正确，第二个维度是状态变化后用户是否授权重选。

维度	Binding Drift 更关注	TRI 更关注
初始绑定	发现错误绑定、复核原始请求、阻止传播	将其作为条件，先隔离到正确绑定
状态变化后	先前实体是否被其他实体替换	保持旧身份还是重新执行查询是否得到用户授权
策略空间	Entity Lock、oracle/learned re-verification	Preserve/Reevaluate 对称控制和编译后的转换授权
正确答案	多以维持或复核原 referent 为中心	相同世界变化下，旧目标和新目标都可能正确

当前已复现 Binding Drift 官方确定性离线 harness 的 25/25 断言，并确认 oracle re-verifier 会读取 gold target；learned self/cross 的 TRI 接口公平适配尚未完成。论文应明确 CTA 不负责修复错误初始绑定，也不声称是更强的通用 Binding Drift defense。真正的新问题是：一个已经解析完成的身份承诺，何时可以被重新转回查询。

8. 当前论文与实验完成度

模块	完成度	现状
问题定义与形式化	90%	TRI、条件分母、mode-blind impossibility、授权不变量已写入正文
主方法	95%	Exact CTA 已锁定；复杂升级未通过 Go/No-Go
v3 主实验	100%	160 条、两模型、消融、对称控制完整
v7 独立复制	100%	240 条、三模型、leave-group-out 与写入审计完整
强基线与消融	95%	完整历史、强语义提醒、mode-only、plan、gate 齐全
人工验证	100%	3 人 100 条盲标、50 条独立改写完成
外部边界	90%	三基准审计、ToolSandbox/AppWorld 工具闭环完成；二次人工审计未做
主论文	85%	可编译 8 页版本：7 页正文 + 1 页参考文献，仍需 reviewer pass
补充材料与 artifact	85%	主要 provenance、原始 run、报告和测试已建立

当前文件状态

- 主 论 文 : paper/AnonymousSubmission2027.tex 与 paper/AnonymousSubmission2027.pdf；当前 8 页。
- 补充材料：paper/supplementary_material.tex 与对应 PDF。
- 证据入口：experiments/tri_artifact/reports/current_claim_provenance.md；主表数字均应沿此回到冻结数据和 raw runs。
- 当前主文已包含 DeepSeek 复制、matched full-history、leave-group-out、SQLite 后果、人工验证和外部零结果。
- 工作区存在尚未提交的最新报告与代码；在最终投稿冻结前还需执行统一测试、匿名包和密钥扫描。

9. 投稿价值、风险与建议师姐重点判断的问题

9.1 当前优势

- 问题直观，最小对可以在很短时间内解释“世界更新”和“目标授权”的区别。
- 严格条件化到正确初始绑定，能把 TRI 与首次选错、工具顺序、动作失效分开。
- v3 发现与 v7 三模型复制形成两阶段证据，且有真实 SQLite 写入后果。
- 人类盲标、独立改写、held-out schema、leave-group-out 降低模板偶然性解释。
- 强完整历史、后置语义提醒、对称策略和表示消融使机制归因相对完整。
- 外部零结果与多指称负结果没有被隐藏，主张边界可信。

9.2 主要风险

风险	具体问题	当前缓解方式
外部有效性	自然 benchmark 未观察到条件 TRI，无法证明真实高频	将贡献定位为 controlled diagnosis + benchmark blind spot
方法新颖性	CTA 本身简单，容易被视为“保存 ID”	强调 Preserve/Reevaluate 对称性、授权形式化、强消融与条件分母
相关工作重叠	与 Binding Drift/Entity Lock 概念接近	用初始绑定 × 转换授权二维地图；不夸大复现范围
合成任务占比	主效应主要来自受控任务	独立改写、新 schema、三模型、SQLite 和工具接口桥梁证据
组合性不足	多刷新、多角色未跨模型稳定解决	明确限制为单刷新、单操作目标，并把 role indexing 留作扩展
强基线例外	Qwen 上 CTA 与后置强提醒总体持平	如实报告，不写成 universally superior

9.3 建议师姐优先给反馈的四个问题

1. 问题和标题是否足够清楚：读完摘要后，能否立即理解“刷新世界不授权更换目标”？

- 2. 相对 Binding Drift 的新颖性是否成立：授权视角和 Preserve/Reevaluate 对称设计是否构成实质区别？
- 3. 主张是否过窄或恰当：应突出“机制发现 + 诊断 benchmark + coverage blind spot”，还是需要更偏系统方法？
- 4. 正文证据取舍是否合理：DeepSeek、完整历史强基线、外部零结果和人工验证中，哪些必须留在 7 页正文？

10. 投稿前剩余工作

优先级	工作	完成标准
P0	逐句核对摘要、主表和结论的数字来源	每个主张可回到 claim provenance、report 和 raw run
P0	完成 reviewer pass 与相关工作定位	不把 oracle 当 learned；不弱化外部 null；术语强度一致
P0	重新生成主表/图并编译主文和 supplement	页数、引用、图表、匿名性与 PDF 元数据全部通过
P0	冻结匿名 artifact	测试通过、clean-room 入口可运行、私有标注映射与 API key 排除
P1	Binding Drift practical reverify author adaptation	仅在接口、公平性和时间允许时做，明确标注非官方 sweep
P1	公开 benchmark 审计第二复核者	复核 near-match 和随机 excluded 样本，报告一致性

建议停手线 不再为了得到正结果扩大同质合成任务或不断修改外部 prompt。现有证据已经足以投稿，新增工作应只解决会改变主文结论的具体缺口。

结语

目前论文最有说服力的部分，不是提出了一个复杂的新 Agent 架构，而是把一个容易被任务成功率掩盖的授权问题拆清楚：选择条件曾经用于找到对象，并不意味着它在刷新后仍有权重新决定对象。受控实验、三模型复制、真实写入和人类验证共同支持这一点；外部零结果和组合负结果则说明该机制不能被夸大为普遍安全缺陷。按照现有进展，论文已经形成完整投稿闭环，下一阶段应以叙事聚焦、相关工作定位和证据审计为主。

附录：阅读原论文时建议先看

- 主论文摘要与 Introduction：快速理解问题、贡献和主张边界。
- Figure 1：理解 Preserve/Reevaluate 的对称结构以及 Generic 与 Lifecycle 的差别。
- Table 1 (controller ladder)：判断收益究竟来自 mode、planning、CTA 还是 gate。
- Results 中 v7 replication、matched full-history 与 human validation：判断鲁棒性和强基线。
- External and Compositional Boundaries + Limitations：判断论文是否诚实处理外部零结果与适用范围。